



Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Actividad Curricular:

IA 4.4 - Procesamiento de Imágenes I

TRABAJO PRÁCTICO N°2

1° Cuatrimestre - Año 2026

Alumnos:
LUJAMBIO, Valentín
MENDEZ, Ignacio
RAMIREZ, Lucas

Fecha:
15/06/2026

Índice

Problema 1 – Detección y clasificación de pastillas	3
1. Marco teórico y resolución	3
2. Desarrollo del script e implementación.....	3
2.1 Segmentación del ROI de la cinta	4
2.2 Detección de pastillas (detectar_pastillas).....	4
2.3 Clasificación por forma y color (clasificar_pastilla)	6
2.4 Reporte y salida (generar_salida)	7
2.5 Conclusión	8
Problema 2 – Detección de patentes	9
1. Marco teórico y resolución.....	9
2. Desarrollo del Script e Implementación.....	9
2.1 Búsqueda de la fila de caracteres (_buscar_fila_patente).....	10
2.2 Detección del bounding box de la placa (detectar_patente).....	11
2.3 Segmentación de caracteres (_detectar_chars)	13
2.4 Conclusión	14

Problema 1 – Detección y clasificación de pastillas

1. Marco teórico y resolución

El objetivo del primer ejercicio es detectar las pastillas que circulan sobre una cinta transportadora industrial y clasificarlas según su tipo, reportando cuántas hay de cada clase. La imagen de entrada presenta un fondo gris (la cinta) sobre el cual descansan cinco tipos de pastillas: redondas blancas, redondas rosas, cuadradas blancas, cápsulas amarillas y cápsulas bicolor (azul-blancas). Además, el marco de la cinta incluye estructuras metálicas y sombras que pueden contaminar la binarización si no se acota previamente la región de trabajo.

La resolución se organizó en cuatro etapas:

- 1. Segmentación del ROI de la cinta:** se aísla rectangularmente el área útil de procesamiento mediante Otsu invertido sobre la escala de grises y operaciones morfológicas (apertura + cierre con núcleos grandes) para desconectar la cinta de los frames metálicos.
- 2. Detección de pastillas:** dentro del ROI, se combina información de brillo (V) y saturación (S) del espacio HSV. La unión de ambos umbrales permite que las pastillas blancas (V alta) y las cápsulas saturadas (S alta) se segmenten correctamente sin confundir la mitad azul con la cinta gris.
- 3. Clasificación:** para cada región se extraen descriptores de forma (circularidad, relación de aspecto, extent) y descriptores de color (porcentaje de píxeles azules, amarillos, rosas y blancos dentro de la máscara). Sobre estos rasgos se aplican reglas lógicas para decidir el tipo.
- 4. Reporte:** se emite por consola el conteo por categoría y se genera una imagen anotada con el bounding box y la etiqueta de cada pastilla.

2. Desarrollo del script e implementación

La solución se implementó en un único script (pastillas.py) dividido en cuatro etapas (A, B, C y D) que se ejecutan en cascada desde el bloque principal. A continuación, se describe cada etapa junto con el código.



Ilustración 1 – Imagen original de la cinta con pastillas

2.1 Segmentación del ROI de la cinta

El primer paso es delimitar el área de la cinta para evitar que el detector confunda elementos del fondo (marco metálico) con pastillas. Se aplica Otsu invertido sobre la imagen en escala de grises: al ser la cinta una superficie oscura uniforme, queda en blanco junto con otras zonas oscuras vecinas. Para desconectarla de esas zonas se usa una apertura morfológica con un núcleo rectangular grande (80x30), de modo que sólo sobrevivan bloques compactos del tamaño de la cinta. Luego se aplica un cierre (55x55) para rellenar los huecos que las pastillas dejan sobre la cinta. Finalmente se queda con la componente conectada de mayor área y se construye una máscara rectangular del bounding box, con una pequeña reducción hacia adentro para evitar tocar los marcos.

```
def segmentar_cinta(img_gray):
    # Umbralizamos la cinta, dejandola blanca
    _, th = cv2.threshold(img_gray, 0, 255,
                          cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    k_open = cv2.getStructuringElement(cv2.MORPH_RECT, (80, 30))
    opened = cv2.morphologyEx(th, cv2.MORPH_OPEN, k_open)
    # Rellenamos los huecos que dejan las pastillas.
    k_close = cv2.getStructuringElement(cv2.MORPH_RECT, (55, 55))
    closed = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, k_close)
    # Nos quedamos con el componente mas grande, que es la cinta
    num, labels, stats, _ = cv2.connectedComponentsWithStats(closed, connectivity=8)
    if num <= 1:
        raise RuntimeError("No se detectó la cinta")
    areas = stats[1:, cv2.CC_STAT_AREA]
    idx = 1 + int(np.argmax(areas))

    x = stats[idx, cv2.CC_STAT_LEFT]
    y = stats[idx, cv2.CC_STAT_TOP]
    w = stats[idx, cv2.CC_STAT_WIDTH]
    h = stats[idx, cv2.CC_STAT_HEIGHT]

    # Devolvemos la mascara rectangular, con un shrink hacia adentro
    # para evitar tocar los frames metálicos del borde.
    pad = max(5, int(0.02 * min(w, h)))
    x0 = x + pad; y0 = y + pad
    x1 = x + w - pad; y1 = y + h - pad
    mask = np.zeros_like(img_gray, dtype=np.uint8)
    mask[y0:y1, x0:x1] = 255
    return mask, (x0, y0, x1 - x0, y1 - y0)
```

2.2 Detección de pastillas (detectar_pastillas)

La segmentación de los objetos se realiza sobre los canales V y S del espacio HSV. Un Otsu directo sobre escala de grises no resulta robusto porque la mitad azul de las cápsulas tiene una intensidad similar a la de la cinta. En cambio, marcando como fondo a los píxeles "claros" (V alto) o "saturados" (S alto, color vivo) se cubren todos los tipos de pastillas. El resultado se restringe a la máscara de la cinta y se limpia con apertura 3x3 y cierre 11x11 para unir las mitades de las capsulas bicolor. Por último, se aplica análisis de componentes conectadas y se filtran candidatos por área mínima (1500px) y dimensión mínima (25px) para descartar ruido.

```

def detectar_pastillas(img_gray, mask_cinta, area_min=1500, min_dim=25, img_hsv=None):
    """
    Punto B: Detecta pastillas combinando brillo (V) y saturación (S). Devolvemos una
    lista de pastillas con sus características.
    """
    if img_hsv is None:
        raise ValueError("Se necesita img_hsv para distinguir cápsulas bicolor")

    V = img_hsv[:, :, 2]
    S = img_hsv[:, :, 1]

    # Umbralizamos V (claros) y S (saturados) con Otsu independiente
    _, th_v = cv2.threshold(V, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    _, th_s = cv2.threshold(S, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    th = cv2.bitwise_or(th_v, th_s)

    # Restringir a la cinta
    th = cv2.bitwise_and(th, mask_cinta)

    # Aplicamos opening y closing para limpiar el ruido
    k_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
    cleaned = cv2.morphologyEx(th, cv2.MORPH_OPEN, k_open)
    k_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))
    cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, k_close)

    # Nos quedamos con los componentes conectados
    num, labels, stats, centroids = cv2.connectedComponentsWithStats(
        cleaned, connectivity=8)

    pastillas = []
    for i in range(1, num):
        area = stats[i, cv2.CC_STAT_AREA]
        if area < area_min:
            continue
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        w = stats[i, cv2.CC_STAT_WIDTH]
        h = stats[i, cv2.CC_STAT_HEIGHT]
        # Descartamos tiras finas (artefactos del borde de la cinta que dan problemas)
        if min(w, h) < min_dim:
            continue
        mask = np.uint8(labels == i) * 255
        pastillas.append({
            'bbox': (x, y, w, h),
            'mask': mask,
            'area': int(area),
            'centroid': (float(centroids[i, 0]), float(centroids[i, 1])),
        })
    return pastillas, cleaned

```

2.3 Clasificación por forma y color (clasificar_pastilla)

A continuación, se adjuntan algunas imágenes de los resultados obtenidos:

Con cada región aislada se calculan dos grupos de descriptores. Los descriptores de forma se obtienen del contorno externo más grande:

- **Circularidad:** $4\pi A / P^2$. Vale aprox 1.0 para círculos y 0.785 para cuadrados. Se usa para distinguir píldoras redondas de cuadradas.
- **Relación de aspecto:** cociente entre el lado largo y el corto del minAreaRect. Un valor ≥ 1.7 indica forma alargada (cápsula).
- **Factor de relleno:** área de la región sobre el área del rectángulo delimitador.

Los descriptores de color se obtienen como fracción de píxeles de la región cuyo H/S cae dentro de rangos predefinidos (azul, amarillo, rosa, blanco). Con estos rasgos se aplican reglas lógicas: primero se decide si es cápsula (alargada o poco circular) y, en ese caso, se diferencia azul-blanca de amarilla por el porcentaje de píxeles azules. Si no es cápsula, se compara la circularidad contra un umbral de 0.88 para separar cuadradas de redondas. Entre las redondas se decide el color por la fracción de rosa.

```
def clasificar_pastilla(mask, img_hsv):
    """
    Punto C: Clasifica la pastilla segun sus características.
    """
    # Tenemos dos características que nos interesan: forma y color, sino obtenemos una
    # forma conocida devolvemos XX
    f = _features_forma(mask)
    c = _features_color(img_hsv, mask)
    if f is None:
        return 'XX'

    # En base a la forma, después vemos el color
    es_capsula = (f['aspect'] >= 1.7) or (f['circularity'] < 0.65)
    if es_capsula:
        if c['pct_blue'] > 0.05:
            return 'CB' # Capsula bicolor azul-blanca
        if c['pct_yellow'] > 0.20:
            return 'CA' # Capsula amarilla
        return 'CB'

    if f['circularity'] < 0.88:
        return 'CC' # Cuadrada blanca

    if c['pct_pink'] > 0.15:
        return 'RR' # Redonda rosa
    return 'RB' # Redonda blanca
```

2.4 Reporte y salida (generar_salida)

La función final recibe la lista de pastillas detectadas y sus etiquetas, genera el conteo por y lo emite por consola. Sobre una copia de la imagen original se dibujan rectángulos verdes alrededor de cada pastilla y se les superpone un identificador único compuesto por la sigla del tipo y un contador (por ejemplo RR1, RR2, CB1, etc.). El resultado se guarda en salida_pastillas.png.

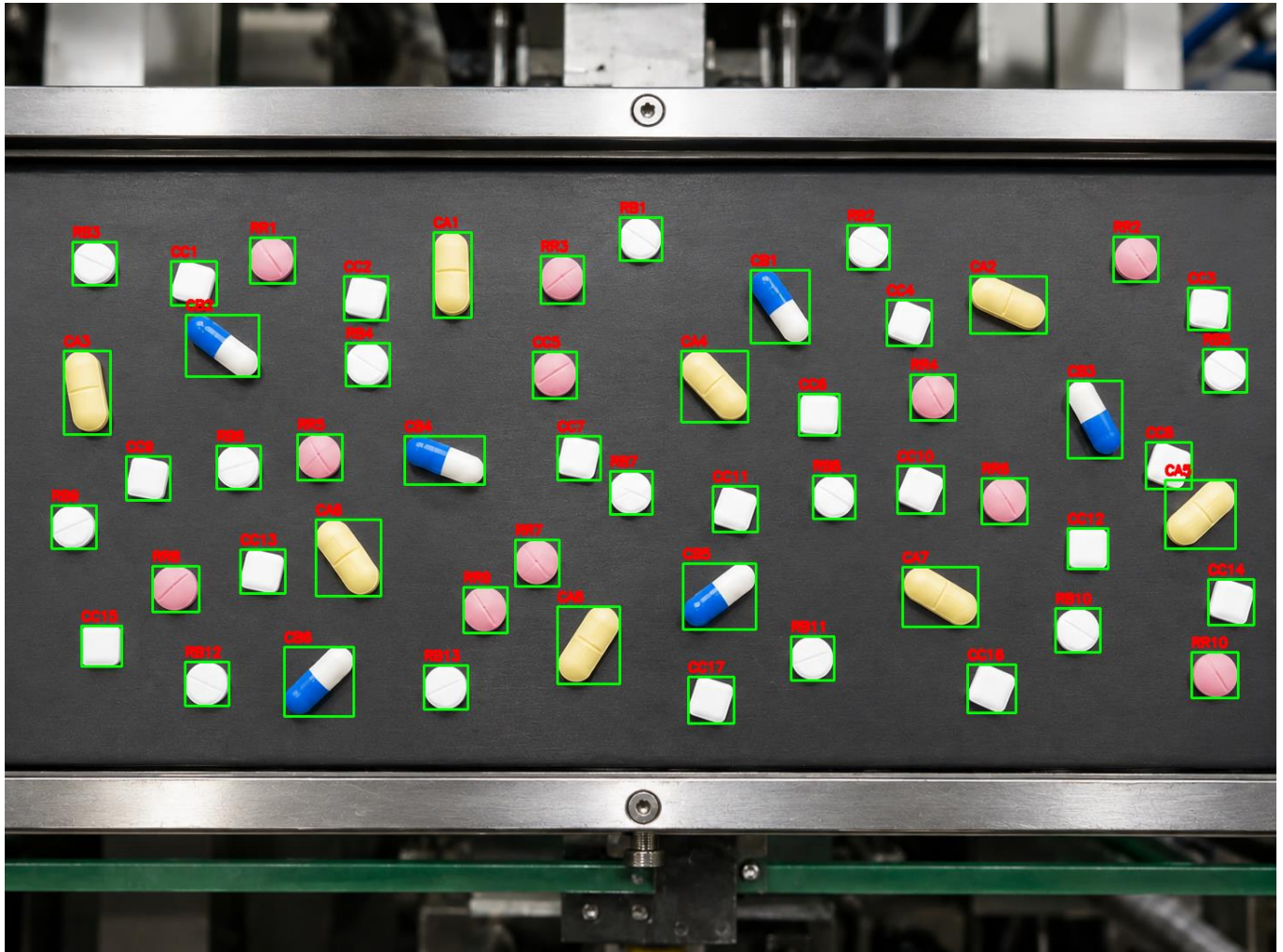


Ilustración 2 - Pastillas delimitadas por bounding box y etiquetas por tipo


```

def generar_salida(img_rgb, pastillas, etiquetas, out_path='salida_pastillas.png'):
    cuenta = Counter(etiquetas)
    print("\n=== Resultado de la clasificación ===")
    total = 0
    for sigla, n in sorted(cuenta.items()):
        nombre = NOMBRES.get(sigla, sigla)
        print(f" {sigla} ({nombre:<22}): {n}")
        total += n
    print(f" {'TOTAL':<28}: {total}")

    # Dibujar etiquetas con id por tipo (RR1, RR2, ...)
    contadores = {}
    out = img_rgb.copy()
    for p, sigla in zip(pastillas, etiquetas):
        contadores[sigla] = contadores.get(sigla, 0) + 1
        etq = f"{sigla}{contadores[sigla]}"
        x, y, w, h = p['bbox']
        cv2.rectangle(out, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(out, etq, (x, max(0, y - 6)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2, cv2.LINE_AA)

    cv2.imwrite(out_path, cv2.cvtColor(out, cv2.COLOR_RGB2BGR))
    imshow(out, color_img=True, title="Pastillas clasificadas")
    print(f"\nImagen guardada en: {out_path}")
    return out

```

2.5 Conclusión

Combinar Otsu sobre los canales V y S del espacio HSV fue clave para que la segmentación funcionara de forma uniforme sobre los cinco tipos de pastilla, en particular para las cápsulas bicolor donde la mitad azul queda invisible si se trabaja únicamente sobre la escala de grises. El acotamiento previo del ROI de la cinta mediante morfología evitó tener que ajustar manualmente umbrales para descartar el marco metálico y los engranajes.

En la etapa de clasificación, apoyarse en descriptores morfológicos simples (circularidad y relación de aspecto) y combinarlos con porcentajes de color por máscara resultó suficiente para discriminar los cinco tipos sin recurrir a un clasificador entrenado, manteniendo la solución totalmente determinística e interpretable.

Problema 2 – Detección de patentes

1. Marco teórico y resolución

El segundo problema consiste en localizar la patente Mercosur dentro de una serie de fotografías tomadas en condiciones reales (vehículos en movimiento, iluminación variable, distinta distancia y ángulos) y, una vez detectada, segmentar los siete caracteres que la componen. A diferencia del primer ejercicio, donde el fondo es controlado, aquí la imagen incluye carrocería, paragolpes, sombras y elementos del entorno que pueden parecerse a una chapa (carteles, calcomanías, logos).

La estrategia adoptada se apoya en propiedades geométricas conocidas:

1. En lugar de buscar el rectángulo blanco de la chapa (poco robusto frente a brillos y reflejos), se busca directamente la fila de 7 caracteres que comparten altura, separación y proporciones compatibles con el estándar Mercosur (400 mm × 130 mm, relación aproximada 3,08).
2. Se aplica binarización adaptativa con varios tamaños de bloque para tolerar cambios de iluminación. Sobre cada binaria se extraen los componentes conectados con forma de carácter (alto suficiente, relación de aspecto angosta) y se agrupan por proximidad vertical.
3. Cada grupo candidato se puntúa con una función que premia: número de caracteres cercano a 7, ancho total del conjunto aprox 5.85·alto, ancho medio de carácter aprox 0.55·alto y espaciado regular entre centros. Se añaden bonificaciones por "fondo blanco" (discriminador clave para diferenciar patente de cartel) y por presencia de franja azul superior (Mercosur).
4. A partir de la fila ganadora se deriva el bounding box de la chapa y se resegmentan los caracteres dentro de ese recorte, ajustando la binarización para conseguir exactamente siete cajas.

2. Desarrollo del Script e Implementación

El código del ejercicio 2 (patentes.py) está organizado en dos grandes etapas: detección y segmentación. La detección se construye sobre funciones auxiliares que separan responsabilidades (extracción de CCs con forma de carácter, scoring de filas, evaluación de blancura del fondo, presencia de franja azul). La segmentación reutiliza esos mismos bloques dentro del recorte de la placa.



Ilustración 3 - Imagen de entrada típica del ejercicio 2: patente AG 678 UA

2.1 Búsqueda de la fila de caracteres (_buscar_fila_patente)

La función central de la detección recorre la imagen completa buscando el mejor candidato a "fila de patente". Primero realiza el contraste local con CLAHE (Contrast Limited Adaptive Histogram Equalization) para que las placas oscurecidas por sombras también queden bien binarizadas. A continuación se aplican tres parametrizaciones de adaptiveThreshold con distintos tamaños de bloque, lo cual asegura que al menos una de ellas funcione frente a la heterogeneidad de las imágenes. Sobre cada binaria se extraen las componentes conectadas con forma de carácter, se agrupan en filas (misma altura y centro y similares), se parten en tramos por separaciones grandes y se puntúan.

```
def _buscar_fila_patente(img):
    """Busca en toda la imagen la fila de ~7 caracteres que mejor matchea la
    geometría de una patente Mercosur. Devolvemos la lista de cajas o []."""
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8))
    gray_eq = clahe.apply(gray)
    Hp = img.shape[0]
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    k_open = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
    mejor_fila, mejor_score = [], -float('inf')
    for bs, C in [(21, 9), (31, 12), (15, 6)]:
        th = cv2.adaptiveThreshold(
            gray_eq, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY_INV, bs, C)
        th = cv2.morphologyEx(th, cv2.MORPH_OPEN, k_open)
        cajas = _ccs_caracteres_like(th, Hp)
        for run, sc in _evaluar_filas(cajas):
            ch = float(np.median([b[3] for b in run]))
            sc += 8.0 * _blancura_fondo(hsv, run) # fondo blanco de la chapa
            if _hay_azul_arriba(hsv, run, ch):
                sc += 2.5 # vemos la franja azul para estar mas seguros
            if sc > mejor_score:
                mejor_score, mejor_fila = sc, run
    return mejor_fila
```

La función _score_fila es la que codifica el conocimiento geométrico de la patente Mercosur. Penaliza linealmente la diferencia contra los valores objetivo (7 caracteres, relación de aspecto del conjunto alrededor de 5.85, ancho medio del carácter aproximadamente 0.55-alto) y castiga la dispersión del paso entre caracteres. De este modo, sin necesidad de buscar la chapa como tal, se identifica la región que cumple con la cadencia visual de una patente.

```

def _score_fila(run, ch):
    n = len(run)
    if n < 5 or n > 9:
        return None
    xs0 = min(b[0] for b in run); xs1 = max(b[0] + b[2] for b in run)
    row_w = xs1 - xs0
    row_aspect = row_w / float(ch) # objetivo ~5.85 para 7 caracteres
    wmed = float(np.median([b[2] for b in run]))
    w_ratio = wmed / ch # objetivo ~0.55
    centros = sorted([b[0] + b[2] / 2.0 for b in run])
    pitches = np.diff(centros)
    pitch_cv = float(np.std(pitches)/(np.mean(pitches) +1e-6)) if len(pitches) else 1.0

    score = 0.0
    score += -1.5 * abs(n - 7)
    score += -1.0 * abs(row_aspect - 5.85)
    score += -3.0 * abs(w_ratio - 0.55)
    score += -2.0 * pitch_cv
    return score

```

2.2 Detección del bounding box de la placa (detectar_patente)

Una vez seleccionada la mejor fila de caracteres, se deriva el bounding box de la chapa expandiendo márgenes proporcionales al alto de los caracteres: aproximadamente el 12% del ancho del run más un carácter de margen lateral, 0.9 altos hacia arriba (para capturar la franja azul Mercosur) y 0.55 altos hacia abajo. Sobre ese recorte generoso se vuelve a ejecutar la detección de caracteres, lo que permite recuperar un 7mo carácter que hubiera quedado fuera del agrupamiento inicial y ajustar el rectángulo final a la extensión real de la fila. Antes de procesar, la imagen se reescala a un ancho fijo de 1100px para uniformar la sensibilidad de los filtros, y al finalizar las coordenadas se remapean a la resolución original.

```

def detectar_patente(img_bgr, debug=False):
    H0, W0 = img_bgr.shape[:2]
    target_w = 1100 # aumentamos la resolucio para patentes lejanas
    scale = target_w / float(W0)
    img = cv2.resize(img_bgr, (target_w, int(H0 * scale)),
                      interpolation=cv2.INTER_AREA)
    fila = _buscar_fila_patente(img)
    if not fila:
        return None, None
    cx0 = min(b[0] for b in fila)
    cy0 = min(b[1] for b in fila)
    cx1 = max(b[0] + b[2] for b in fila)
    cy1 = max(b[1] + b[3] for b in fila)
    ch = float(np.median([b[3] for b in fila]))
    row_w = cx1 - cx0
    mx = int(0.12 * row_w + ch)
    my_top = int(0.90 * ch) # franja azul arriba
    my_bot = int(0.55 * ch)
    bx0 = max(0, cx0 - mx); by0 = max(0, cy0 - my_top)
    bx1 = min(img.shape[1], cx1 + mx); by1 = min(img.shape[0], cy1 + my_bot)
    crop = img[by0:by1, bx0:bx1]
    # Comparamos con la geometría de la patente para mas precision
    cajas, _ = _detectar_chars(crop)
    if len(cajas) >= 4:
        dx0 = min(c[0] for c in cajas); dy0 = min(c[1] for c in cajas)
        dx1 = max(c[0] + c[2] for c in cajas); dy1 = max(c[1] + c[3] for c in cajas)
        chh = dy1 - dy0
        px = max(int(0.06 * (dx1 - dx0)), int(0.3 * chh))
        py_top = int(0.75 * chh)
        py_bot = int(0.40 * chh)
        x = bx0 + max(0, dx0 - px); y = by0 + max(0, dy0 - py_top)
        x1 = bx0 + dx1 + px; y1 = by0 + dy1 + py_bot
    else:
        px = int(0.05 * row_w)
        x = max(0, cx0 - px); y = max(0, cy0 - int(0.70 * ch))
        x1 = min(img.shape[1], cx1 + px); y1 = min(img.shape[0], cy1 + int(0.40 * ch))
    if debug:
        vis = img.copy()
        for b in fila:
            cv2.rectangle(vis, (b[0], b[1]),
                          (b[0] + b[2], b[1] + b[3]), (0, 255, 0), 1)
            cv2.rectangle(vis, (x, y), (x1, y1), (0, 0, 255), 2)
            imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB), color_img=True,
                  title="A - fila de caracteres + bbox patente")
    x0o = int(x / scale); y0o = int(y / scale)
    x1o = int(x1 / scale); y1o = int(y1 / scale)
    patente = img_bgr[y0o:y1o, x0o:x1o].copy()
    return patente, (x0o, y0o, x1o - x0o, y1o - y0o)

```

2.3 Segmentación de caracteres (_detectar_chars)

Dentro del recorte de la placa se ejecuta una etapa final dedicada a obtener exactamente siete cajas. Para ganar robustez se prueba un pequeño "barrido" de cinco configuraciones de binarización (tres parametrizaciones de adaptiveThreshold gaussiano, una de media y un Otsu de respaldo). Cada binaria se limpia con una apertura morfológica y se extraen las componentes con forma plausible de carácter. La función _linea_dominante se queda con el grupo más numeroso de cajas que comparten altura y centro y similares (descartando barras verticales de la carrocería o reflejos). Se elige la configuración cuyo recuento se acerca más a 7. Si tras el barrido aún quedan más de siete cajas, se conservan las siete cuyo alto es más cercano a la mediana del grupo.

```
def _detectar_chars(placa_bgr, debug=False):
    h, w = placa_bgr.shape[:2]
    if h == 0 or w == 0:
        return [], placa_bgr
    h_target = 130
    scale = h_target / float(h)
    placa=cv2.resize(placa_bgr,(max(1,int(w*scale)),h_target),interpolation=cv2.INTER_CUB
    IC)
    gray = cv2.cvtColor(placa, cv2.COLOR_BGR2GRAY)
    clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8)) # Realizamos un CLAHE
    gray_eq = clahe.apply(gray) #para mejorar el contraste de la imagen
    # Probamos varias configuraciones de adaptiveThreshold + Otsu (fallback)
    configs=[('gauss', 21, 8),('gauss',31,10),('gauss',15,6),('mean',21,8),('otsu',0,0)]
    k_open = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
    mejor_cajas, mejor_th, mejor_score = [], None, -float('inf')
    for tipo, bs, C in configs:
        if tipo == 'gauss':
            th = cv2.adaptiveThreshold(
                gray_eq,255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY_INV, bs,C)
        elif tipo == 'mean':
            th = cv2.adaptiveThreshold(
                gray_eq, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY_INV, bs, C)
        else:
            _, th=cv2.threshold(gray_eq, 0, 255, cv2.THRESH_BINARY_INV+ cv2.THRESH_OTSU)
        th = cv2.morphologyEx(th, cv2.MORPH_OPEN, k_open)
        cajas = _linea_dominante(_extraer_cajas_chars(th))
        score = -abs(len(cajas) - 7)
        if score > mejor_score:
            mejor_score, mejor_cajas, mejor_th = score, cajas, th
            if score == 0:
                break
    cajas = mejor_cajas
    if len(cajas) > 7:
        h_med = float(np.median([b[3] for b in cajas]))
        cajas = sorted(cajas, key=lambda b: abs(b[3] - h_med))[:7]
    cajas.sort(key=lambda b: b[0])
    if debug and mejor_th is not None:
        imshow(mejor_th, title="B.1 - mejor binarización")
    cajas_orig = [(int(x / scale), int(y / scale),int(cw / scale), int(ch / scale))
                  for (x, y, cw, ch) in cajas]
    return cajas_orig, placa
```

2.4 Conclusión

Apuntar primero a la fila de caracteres y derivar la chapa a partir de ella resultó mucho más estable que intentar encontrar el rectángulo blanco de la patente con técnicas clásicas de detección de bordes. La función de scoring incorpora el conocimiento del dominio (proporciones oficiales de la patente Mercosur, cantidad y disposición de los caracteres) de forma explícita y permite descartar carteles y logos comerciales que tendrían un perfil de binarización parecido pero geometría incompatible.

El uso conjunto de CLAHE para el realce de contraste y de varias parametrizaciones de adaptiveThreshold absorbió la mayor parte de la variabilidad de iluminación entre las doce imágenes. Los bonus por "blancura de fondo" y "franja azul superior" funcionaron como un discriminador semántico final que privilegió a las patentes reales cuando otras regiones de la imagen presentaban una cadencia de caracteres similar.

img_1.jpg

Original con patente detectada



patente - 7 caracteres segmentados

